INFO 510 Bayesian Modelling and Inference

Lecture 13 – Metropolis-Hastings

(MCMC algorithm)

Dr. Kunal Arekar
College of Information
University of Arizona, Tucson

November 3, 2025

Previous two lectures…

Markov Chain

Markov chain is a process that consists of a finite number of states and some known probabilities $\boldsymbol{p_{ij}}$, where $\boldsymbol{p_{ij}}$ is the probability of moving from state ***i*** to state ***j***.
Transition probability (transition matrix)

Markov Property and Memorylessness

The key feature of Markov Chains is that the future state depends only on the current state, not on the sequence of events that preceded it. This property is called the Markov Property.

$$P(X_{t+1} = s \mid X_t = s_t , \cancel{X_{t-1} = s_{t-1}, \ldots, X_0 = s_0}) = P(X_{t+1} = s \mid X_t = s_t)$$

distribution of X_{t+1}

depends on X_t

whatever happened before time t doesn't matter.

Nestlé

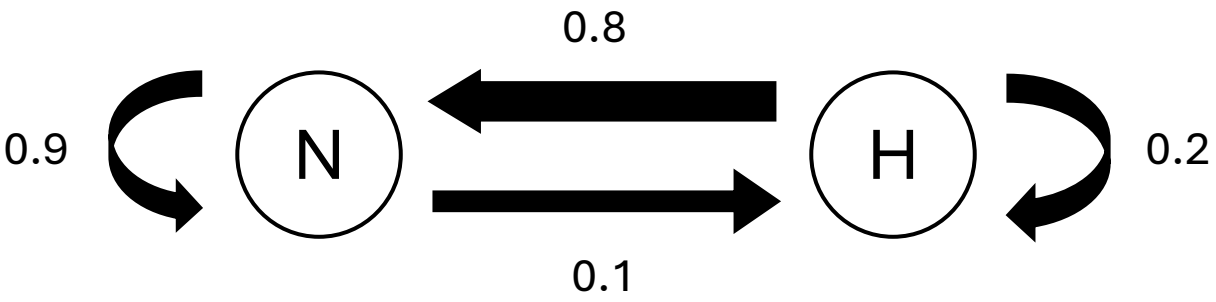
HERSHEY
THE HERSHEY COMPANY

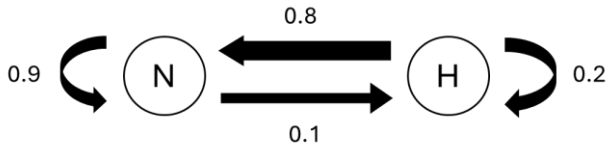
Some chocolate

Sum chocolate

Nestle = 0.6

Hershey = 0.4





		t + 1	
		N	H
t	N	0.9	0.1
	H	0.8	0.2

Transition matrix

Nestle = 0.6

Hershey = 0.4

At time t + 1, what market share will Nestle and Hershey have?

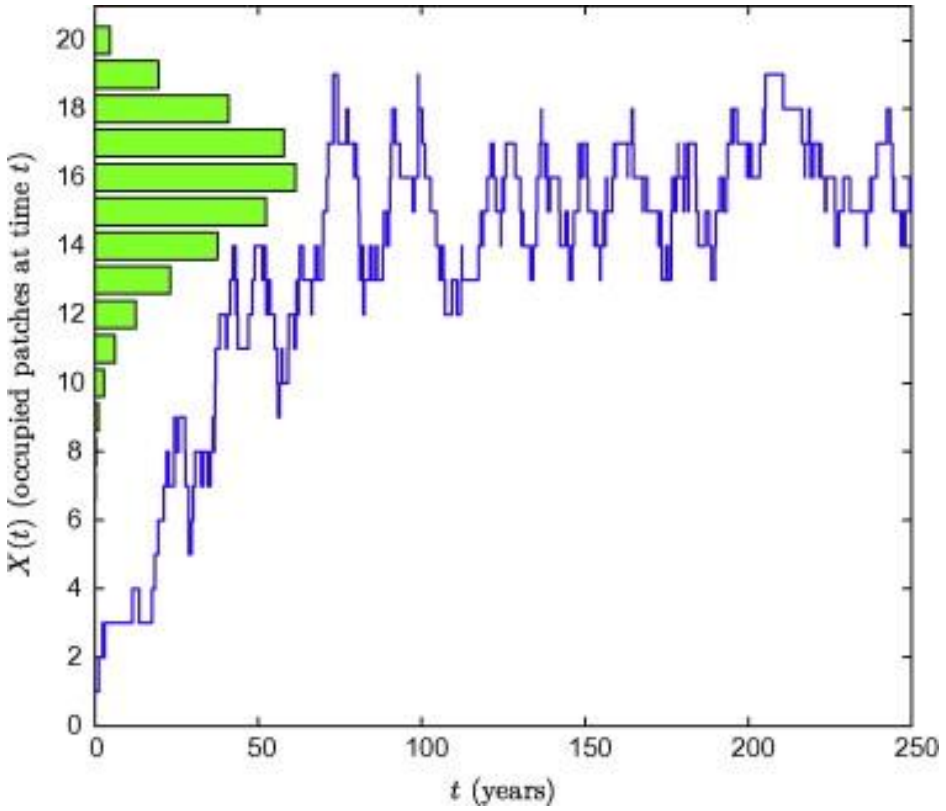
	N	H
t	0.6	0.4
t + 1	0.86	0.14
t + 2	0.886	0.114

N (t + 1) = (0.6 * 0.9) + (0.4 * 0.8)
= 0.54 + 0.32 = 0.86

H (t + 1) = (0.4 * 0.2) + (0.6 * 0.1)
= 0.08 + 0.06 = 0.14

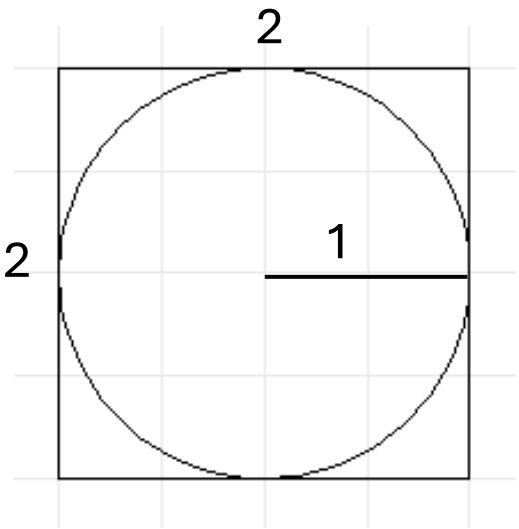
Stationary Distribution - Where the System Settles

After many steps, a Markov chain may reach a state where the probabilities stop changing

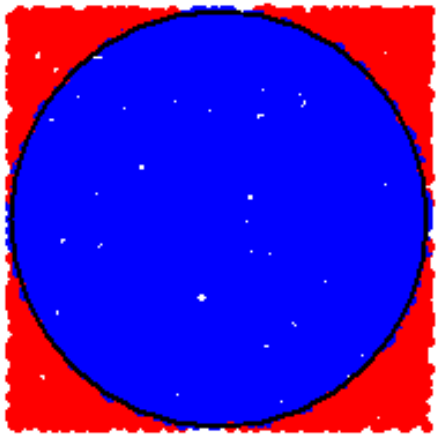


van Doorn and Pollett, 2013. European Journal of Operational Research. Volume 230, Issue 1. Pages 1-14

Monte Carlo methods



Prob of a random point inside the circle = π/4 = **0.7853982**



N (10) = 0.9
N (100) = 0.81
N (10000) = 0.7903

```
N <- 1000000
circ <- 0

for (i in 1:N) {
                x <- runif(1, -1, 1)
                y <- runif(1, -1, 1)

                if (x^2 + y^2 <= 1) {
                        circ <- circ + 1
                        }
                }
```

test <- circ/N = **0.785313**
~ 3 secs

Monte Carlo methods

Problems where an exact solution is difficult or impossible to compute directly

These problems include high-dimensional integrals, complex probabilistic models, or large-scale simulations

For e.g.

Simulating stock prices - how a portfolio of stocks will behave under different market conditions, randomly varying interest rates, stock prices, and other factors (finance)

simulate complex systems - how particles move and interact (physics)

Use Monte Carlo methods which solves problems through random sampling

useful when dealing with complex systems where a deterministic solution (i.e., one where every step is fully calculated) is either impossible or computationally expensive

Two key algorithms

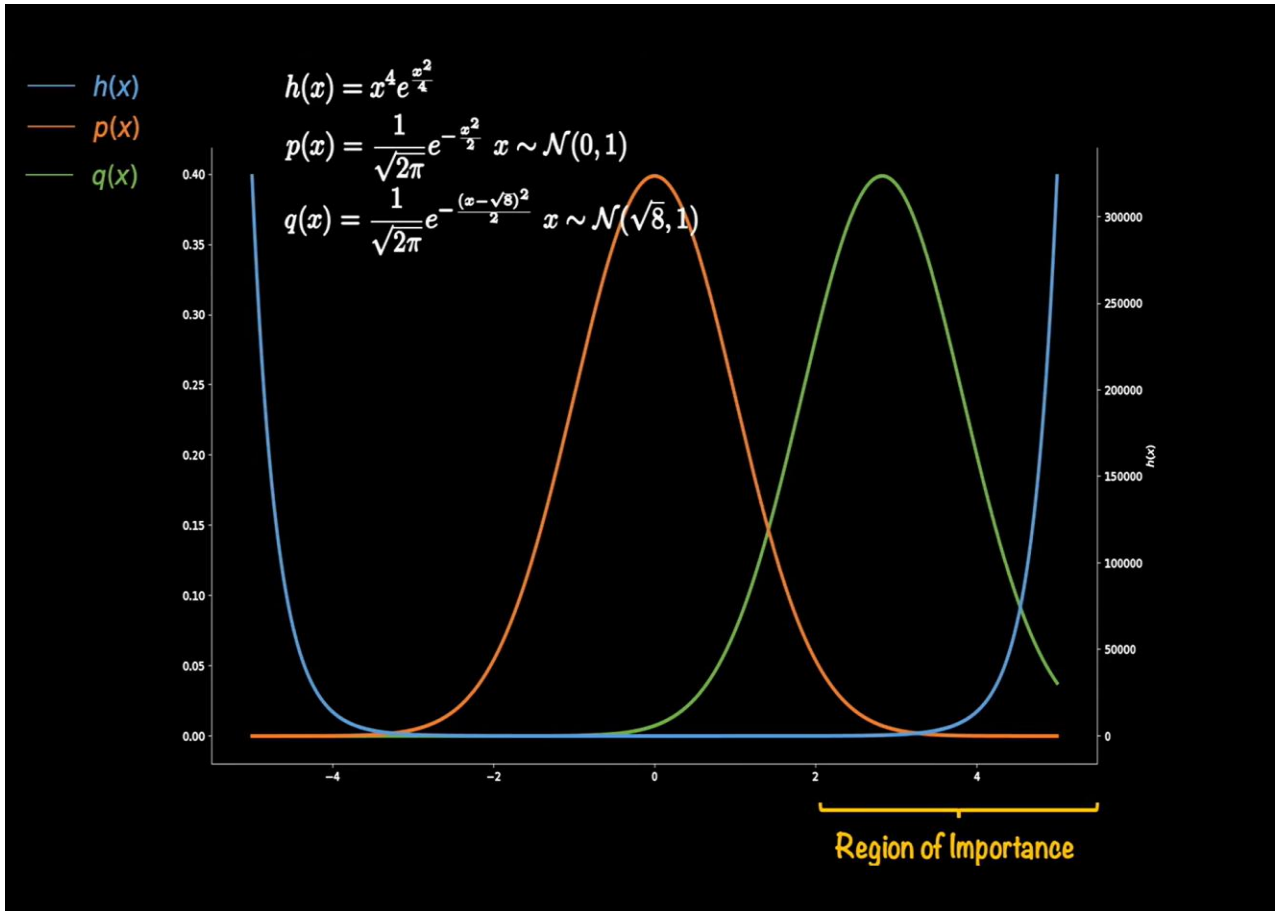
Importance sampling

Sometimes, sampling directly from a target distribution (e.g., a posterior distribution) is difficult
Importance sampling allows us to sample from an easier distribution and re-weight the samples to approximate the target distribution

Rejection sampling

Propose samples from an easier distribution
Accept or reject each sample based on how well it fits the target distribution

Importance Sampling



Where Does the Easier Distribution q(x) Come From?

Where Does the Easier Distribution q(x) Come From?

It doesn't just appear magically, it's chosen strategically

The choice depends on the specific problem at hand

Some common guidelines to choose q(x):

Match the Shape of f(x) in "Important" regions

Simpler Functional Forms – q(x) should have a simpler mathematical form that allows easy sampling

Tail Behavior – if f(x) has long tail, you would want q(x) to have long tail

pick one that minimizes the variance of the estimate

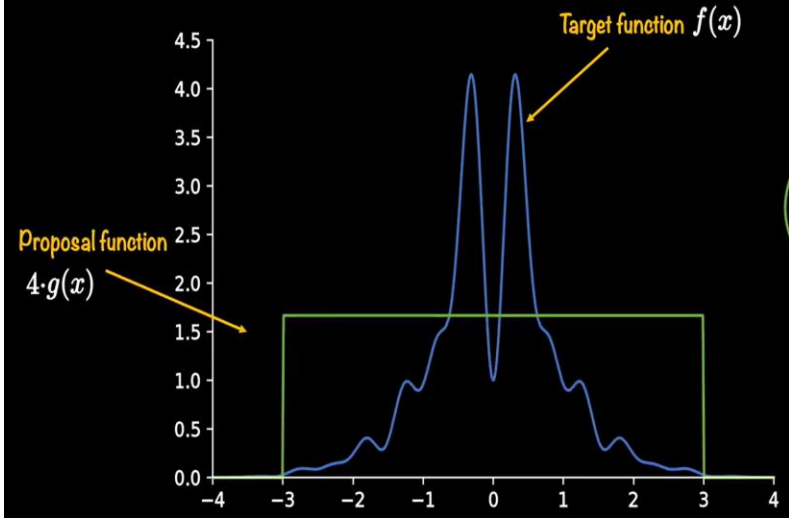
Rejection sampling

Propose samples from an easier distribution
Accept or reject each sample based on how well it fits the target distribution

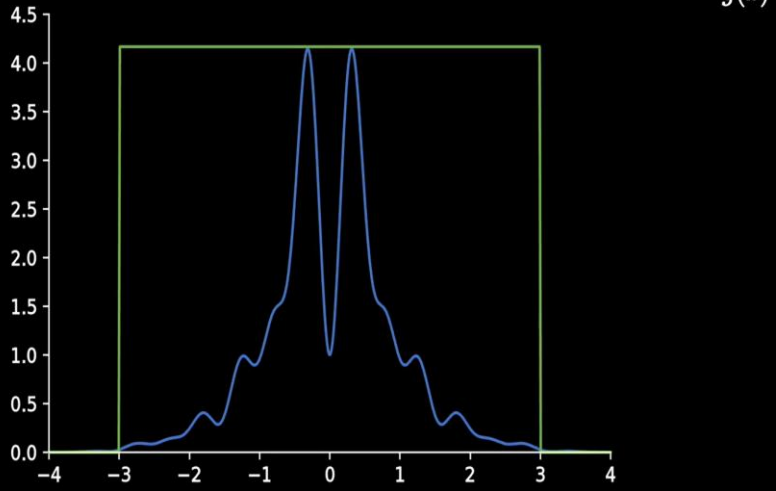
Rejection sampling or Accept – Reject sampling

- Select a proposal distribution
- Scale it
- Use the acceptance criteria that involves randomness to make a decision

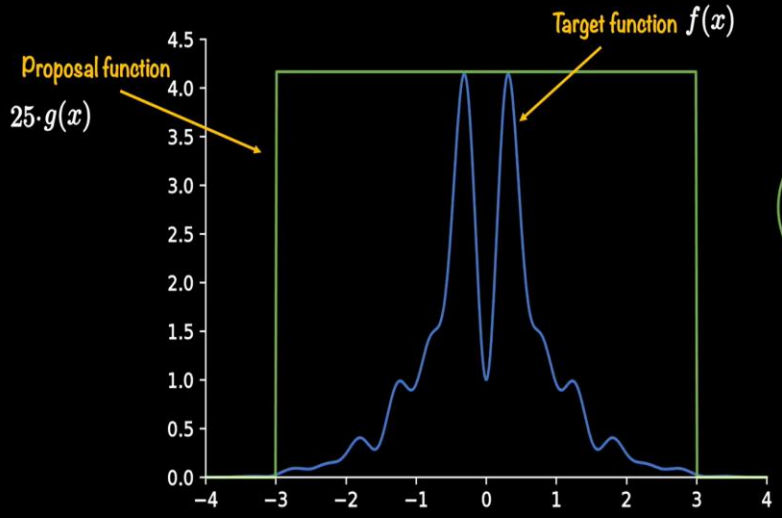
$$f(x) = e^{-x^2/2} \cdot (\sin^2(6+x) + 3 \cdot \cos^2(x) \cdot \sin^2(4x) + 1)$$



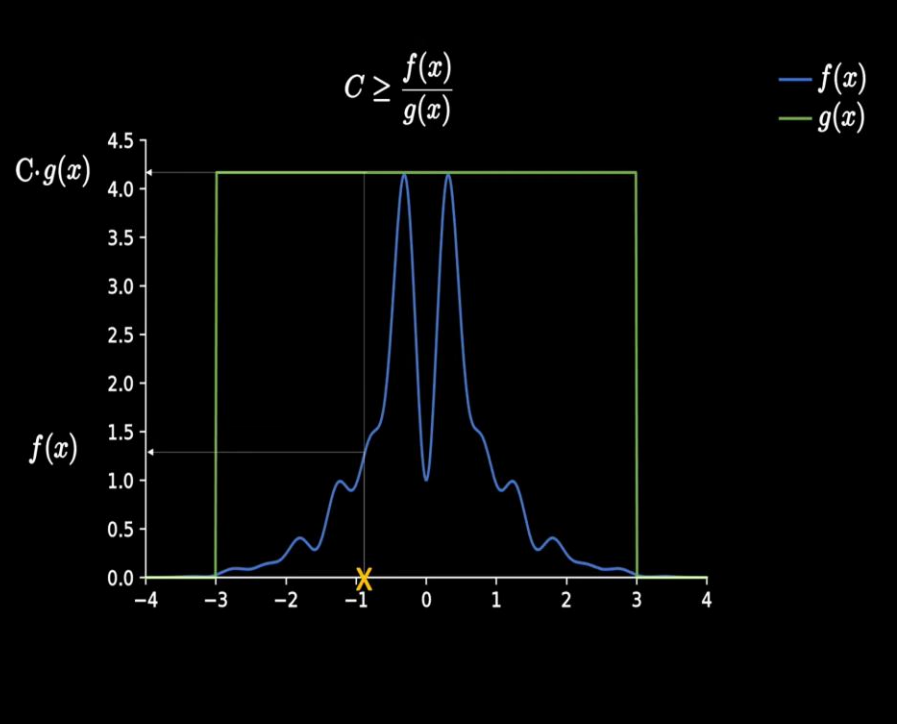
$$x \sim \mathcal{U}(-3, 3)$$
$$g(x) = \frac{1}{b-a}$$

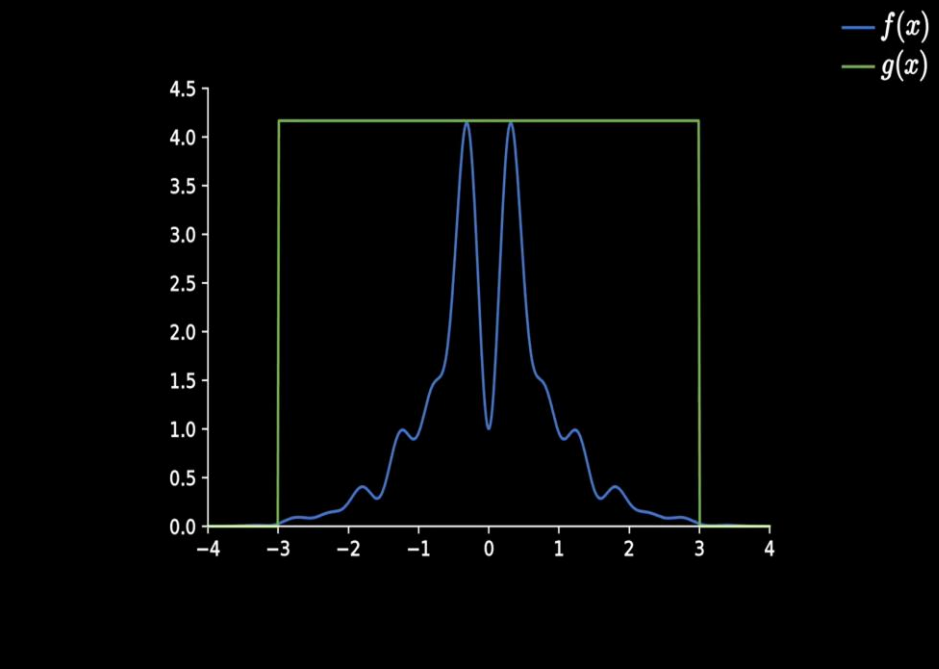


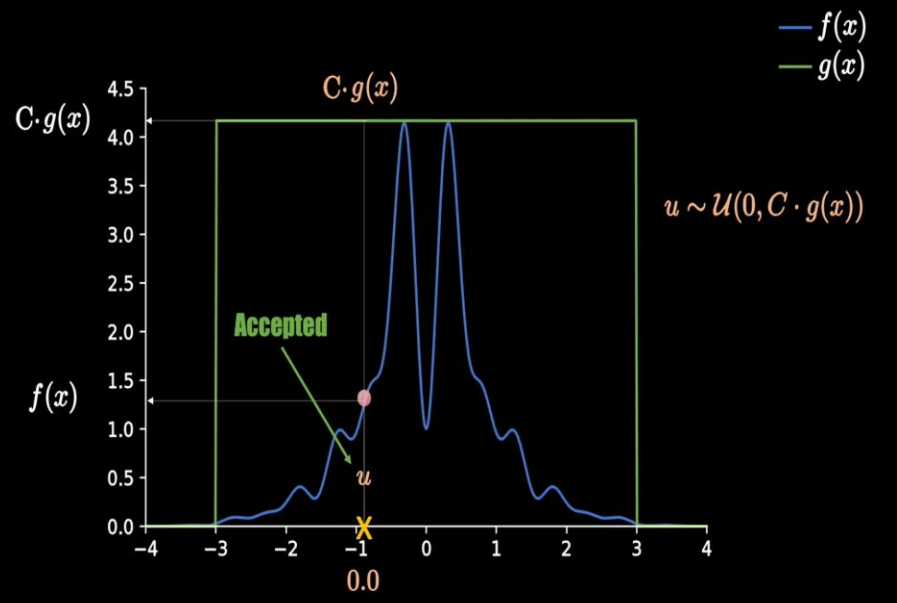
$$f(x) = e^{-x^2/2} \cdot (\sin^2(6+x) + 3 \cdot \cos^2(x) \cdot \sin^2(4x) + 1)$$

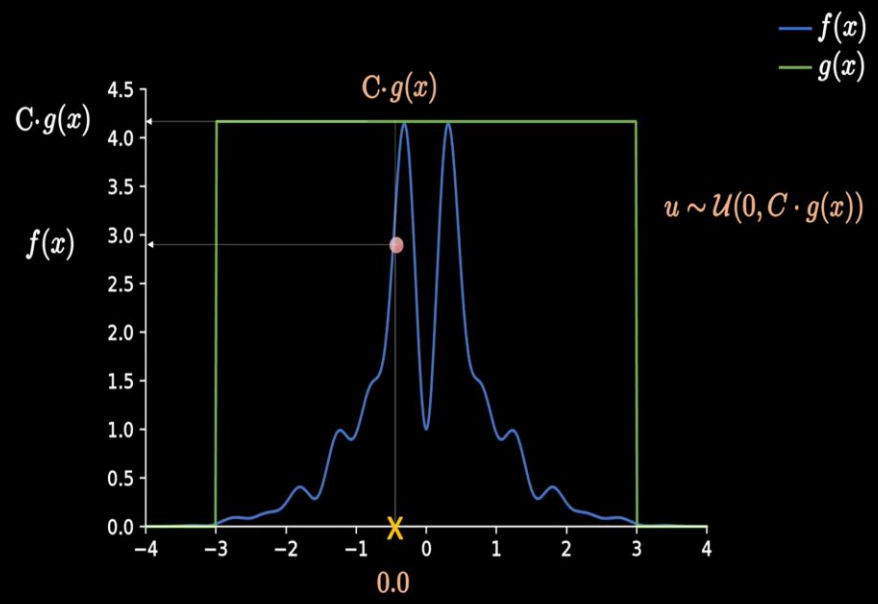


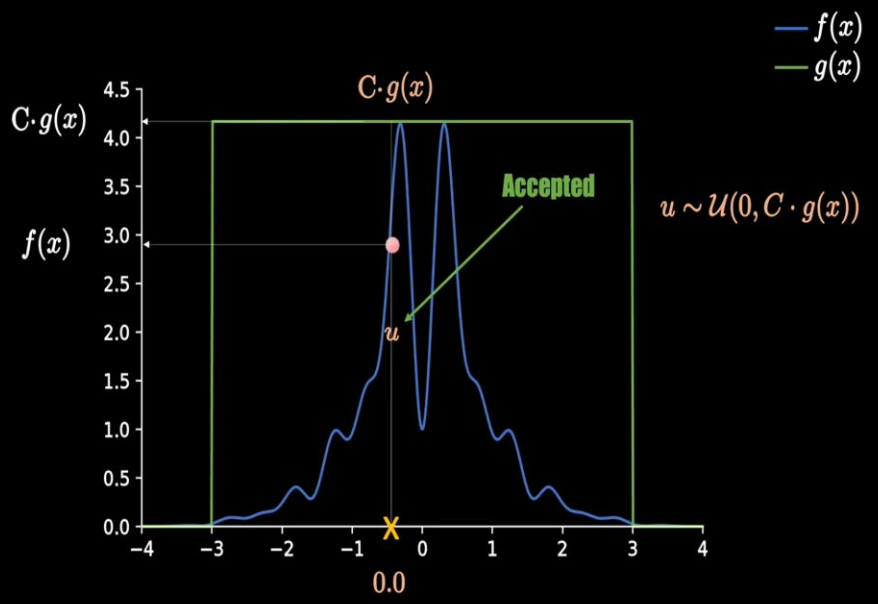
$$x \sim \mathcal{U}(-3, 3)$$
$$g(x) = \frac{1}{b-a}$$


$C \geq \frac{f(x)}{g(x)}$
$f(x)$
$g(x)$
$C \cdot g(x)$
$f(x)$
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−4 −3 −2 −1 0 1 2 3 4


$f(x)$
$g(x)$
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−4 −3 −2 −1 0 1 2 3 4

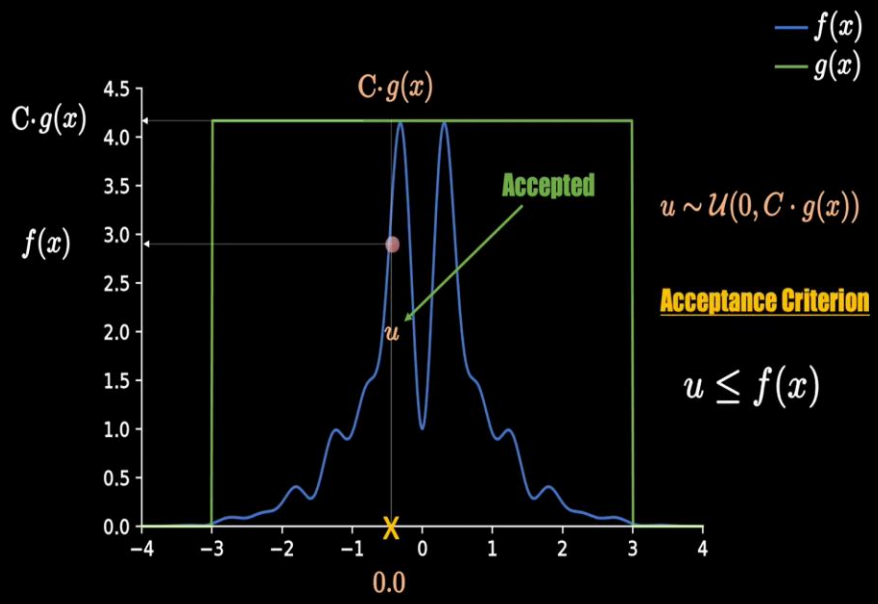

$f(x)$
$g(x)$
$C \cdot g(x)$
$C \cdot g(x)$
$u \sim \mathcal{U}(0, C \cdot g(x))$
Accepted
$f(x)$
u
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−4 −3 −2 −1 0 1 2 3 4
0.0

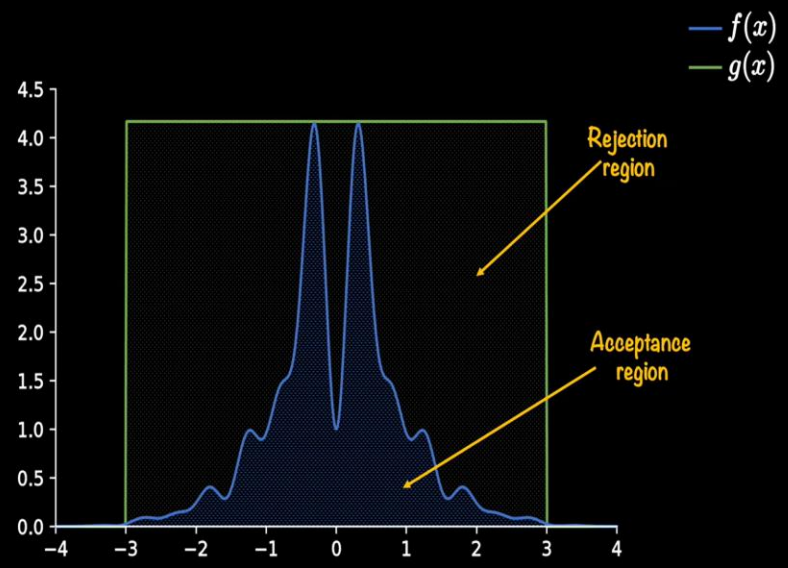


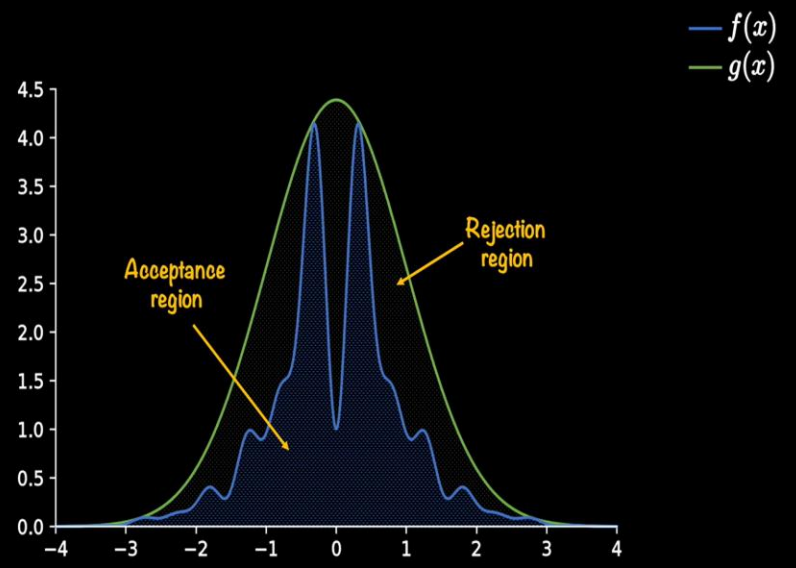


Rejection sampling –

- Select a proposal distribution
- Scale it
- Use the acceptance criteria that involves randomness to make a decision







Run it many times ~1 mill

Acceptance rate = 23.5%

Acceptance rate = 53.63%

Markov Chain Monte Carlo (MCMC)

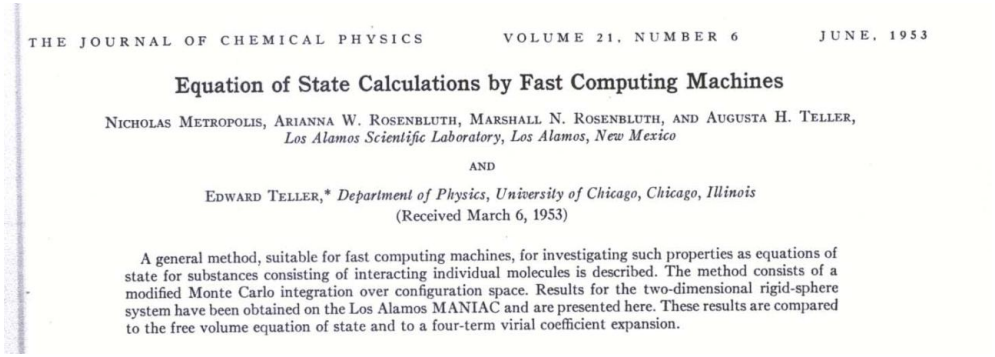
Is a set of algorithms to sample from a probability distribution such that the samples match the distribution statistics.

- Markov - screening assumption, the next sample is only dependent on the previous sample
- Chain - the samples form a sequence often demonstrating a transition from burn-in chain with inaccurate statistics and equilibrium chain with accurate statistics
- Monte Carlo - use of Monte Carlo simulation, random sampling from a statistical distribution

Why is this useful?

- we often don't have the target distribution, it is unknown
- but we can sample with the correct frequencies with other form of information such as conditional probability density functions

Metropolis-Hastings Algorithm



THE JOURNAL OF CHEMICAL PHYSICS VOLUME 21, NUMBER 6 JUNE, 1953

Equation of State Calculations by Fast Computing Machines

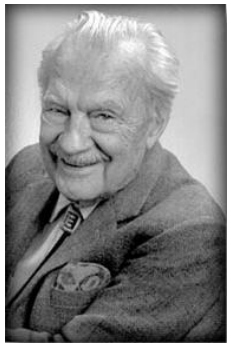
NICHOLAS METROPOLIS, ARIANNA W. ROSENBLUTH, MARSHALL N. ROSENBLUTH, AND AUGUSTA H. TELLER,
Los Alamos Scientific Laboratory, Los Alamos, New Mexico

AND

EDWARD TELLER,* *Department of Physics, University of Chicago, Chicago, Illinois*

(Received March 6, 1953)

A general method, suitable for fast computing machines, for investigating such properties as equations of state for substances consisting of interacting individual molecules is described. The method consists of a modified Monte Carlo integration over configuration space. Results for the two-dimensional rigid-sphere system have been obtained on the Los Alamos MANIAC and are presented here. These results are compared to the free volume equation of state and to a four-term virial coefficient expansion.



Probably the most popular MCMC algorithm

Developed as an extension of the basic Metropolis algorithm (1953)

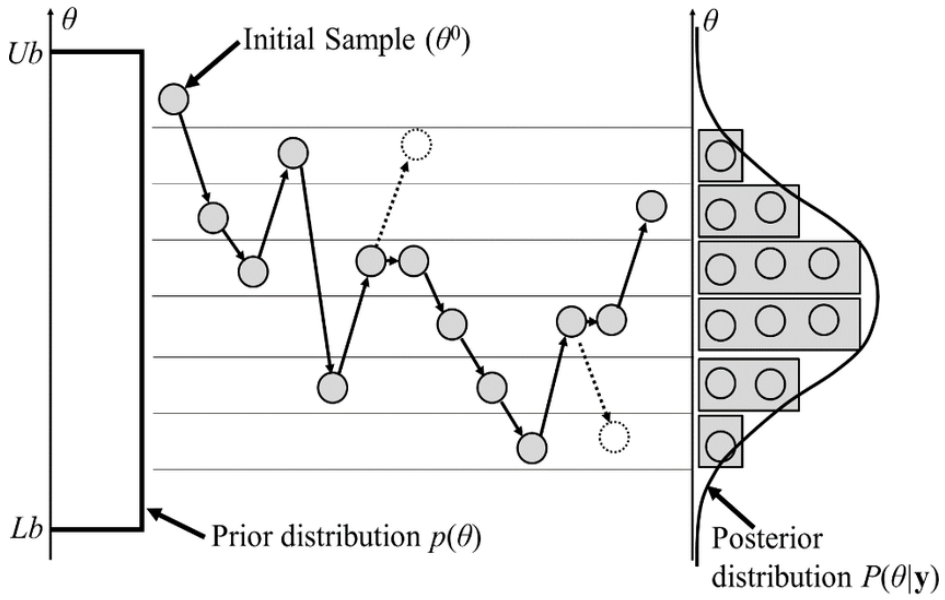
Arianna Wright wrote the first full implementation of the MCMC method.

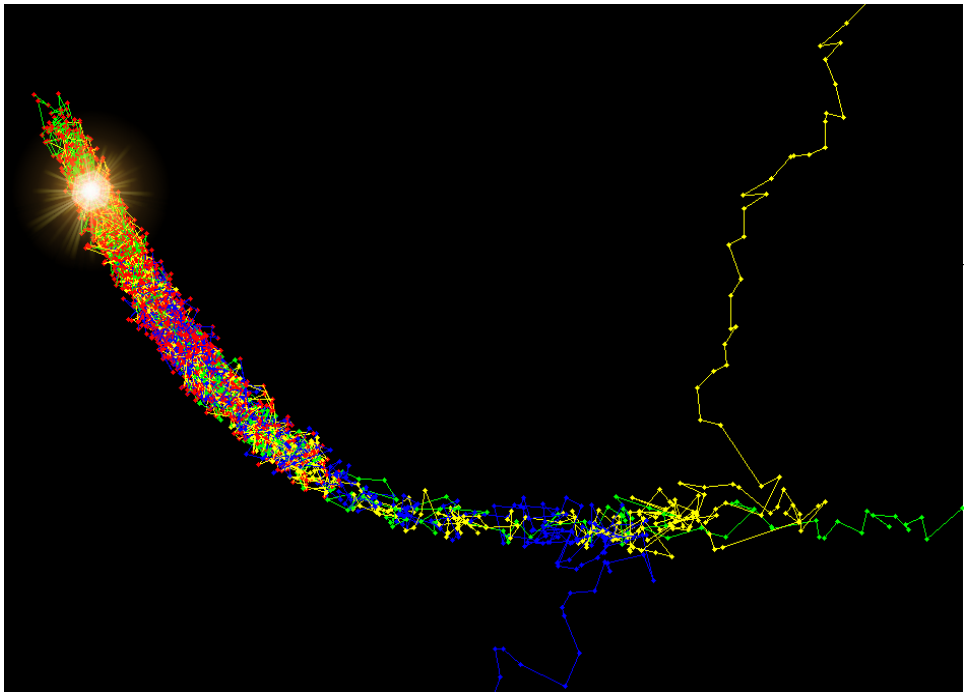
Hastings in 1970, extended the solution to non-symmetric proposal

Samples from a target distribution, $\pi(x)$

Combines a proposal distribution with an acceptance-rejection mechanism to create a chain of samples that converge to the target distribution

Metropolis-Hastings Algorithm





Metropolis–Hastings and other MCMC algorithms are generally used for sampling from multi-dimensional distributions, especially when the number of dimensions is high.

Metropolis-Hastings Algorithm

Steps of the Algorithm

Initialize - start with an arbitrary initial point x_t

Propose - from the current state x_t, propose a new state x' using a proposal distribution $q(x'|x_t)$

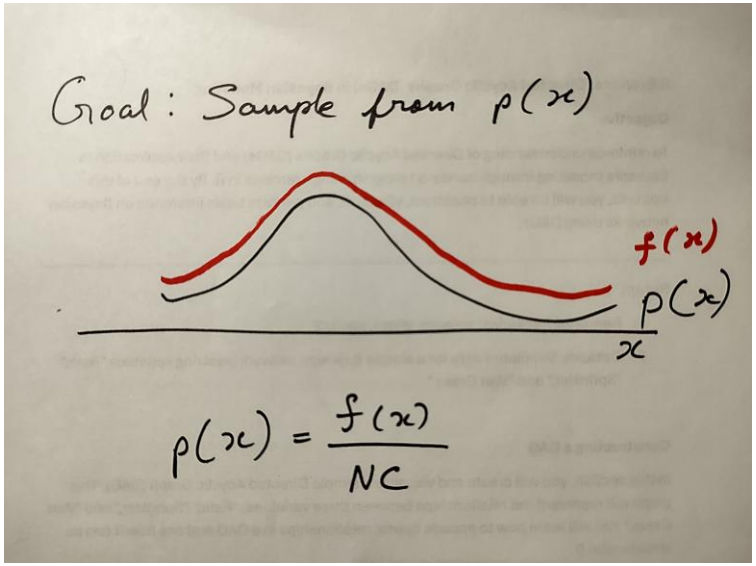
Compute the acceptance probability

$$\alpha = \min\left(1, \frac{\pi(x')q(x_t|x')}{\pi(x_t)q(x'|x_t)}\right)$$

Accept or Reject -
With probability α, accept the new state x' (set x_{t+1}=x')

Otherwise, stay at the current state $x_{t+1} = x_t$

Iterate this process to generate a chain of samples

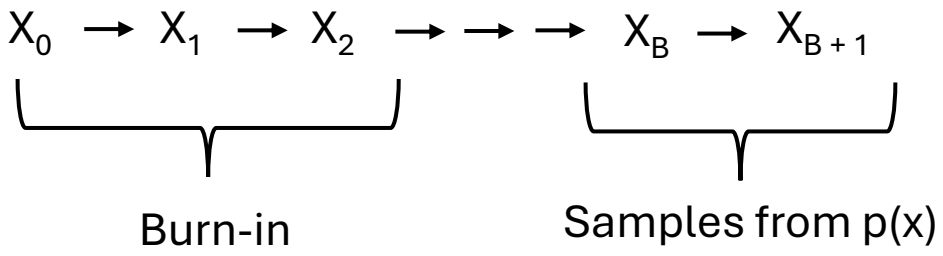


We don't always know the exact form of p(x)

We only know this numerator term f(x)

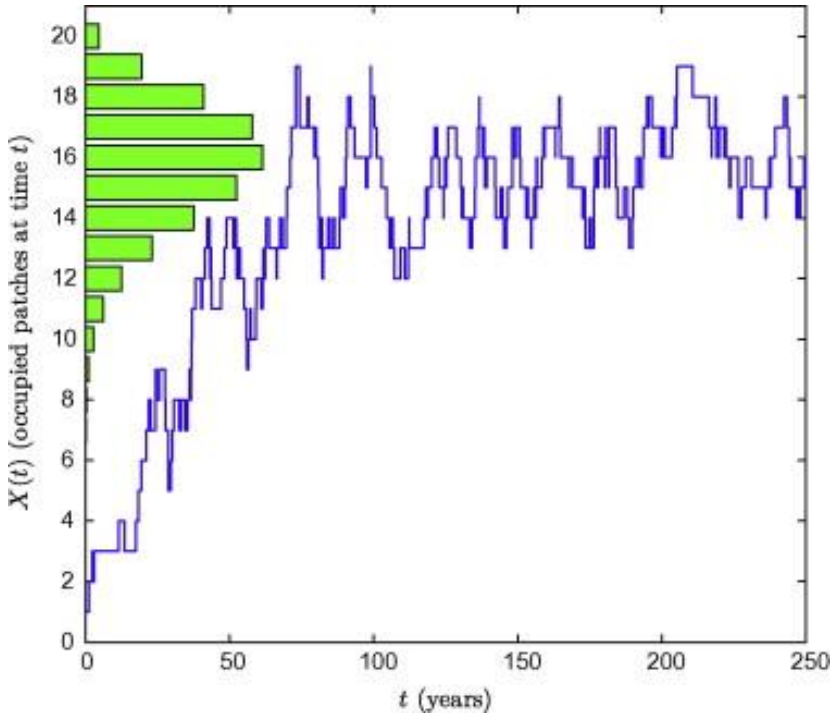
Can we use only f(x) to get samples from p(x)?

Idea behind MCMC



How to estimate the transition probabilities from one state to another?

Stationary Distribution - Where the System Settles



After many steps, a Markov chain may reach a state where the probabilities stop changing

$X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow \rightarrow \rightarrow X_B \rightarrow X_{B+1}$

Burn-in

Samples from p(x)

How to estimate the transition probabilities from one state to another?

van Doorn and Pollett, 2013. European Journal of Operational Research. Volume 230, Issue 1. Pages 1-14

Metropolis-Hastings Algorithm

Suppose that the current state of the Markov chain is x_n and we want to generate x_{n+1}

first stage is to generate a candidate x*

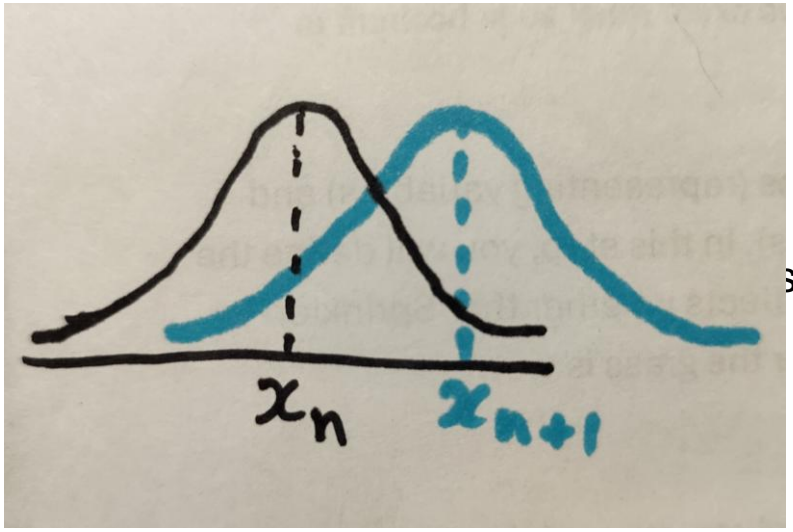
x* is generated from the proposal distribution that we already know how to sample from

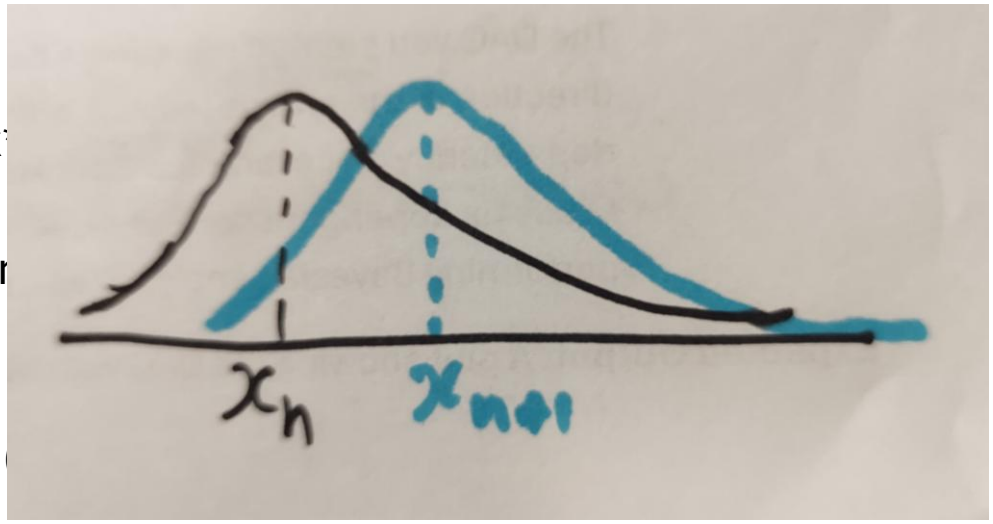
Denote this proposal distribution as $q(x^*|x_n)$.

The proposal distribution could typically be a normal distribution centered on the current state x_n

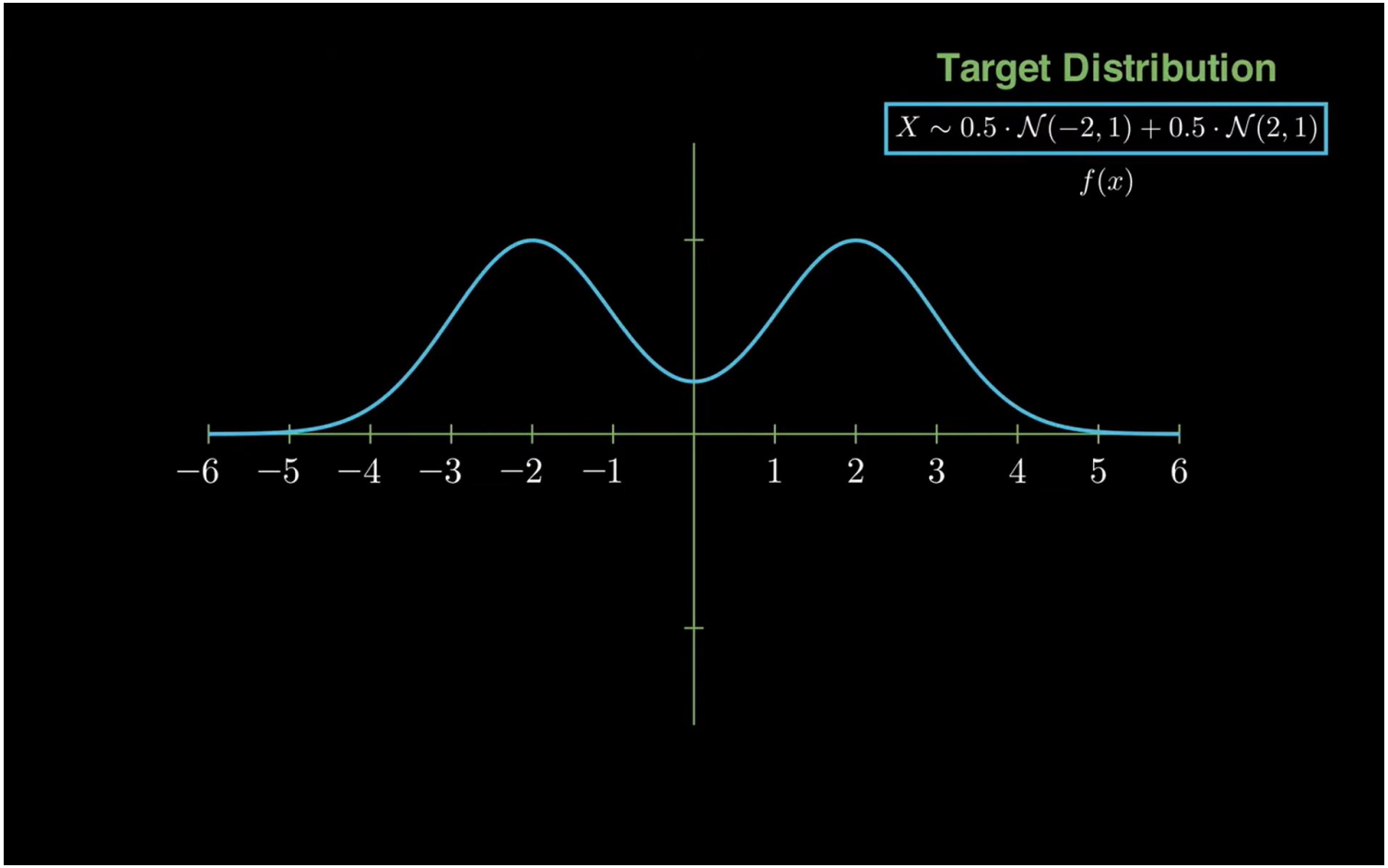
Metropolis-Hastings Algorithm

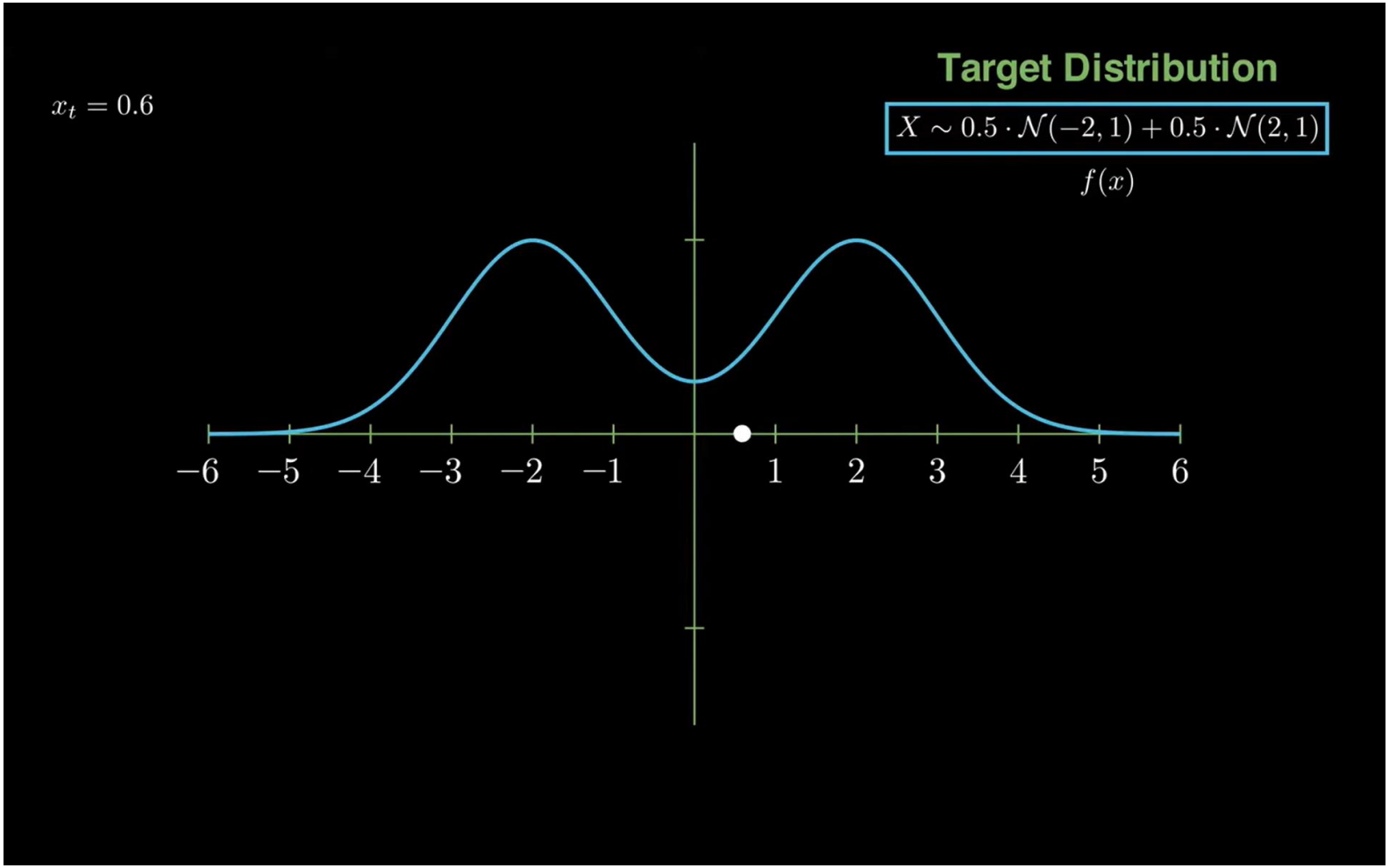
Suppose that the current state of the Markov chain is x_n and we want to

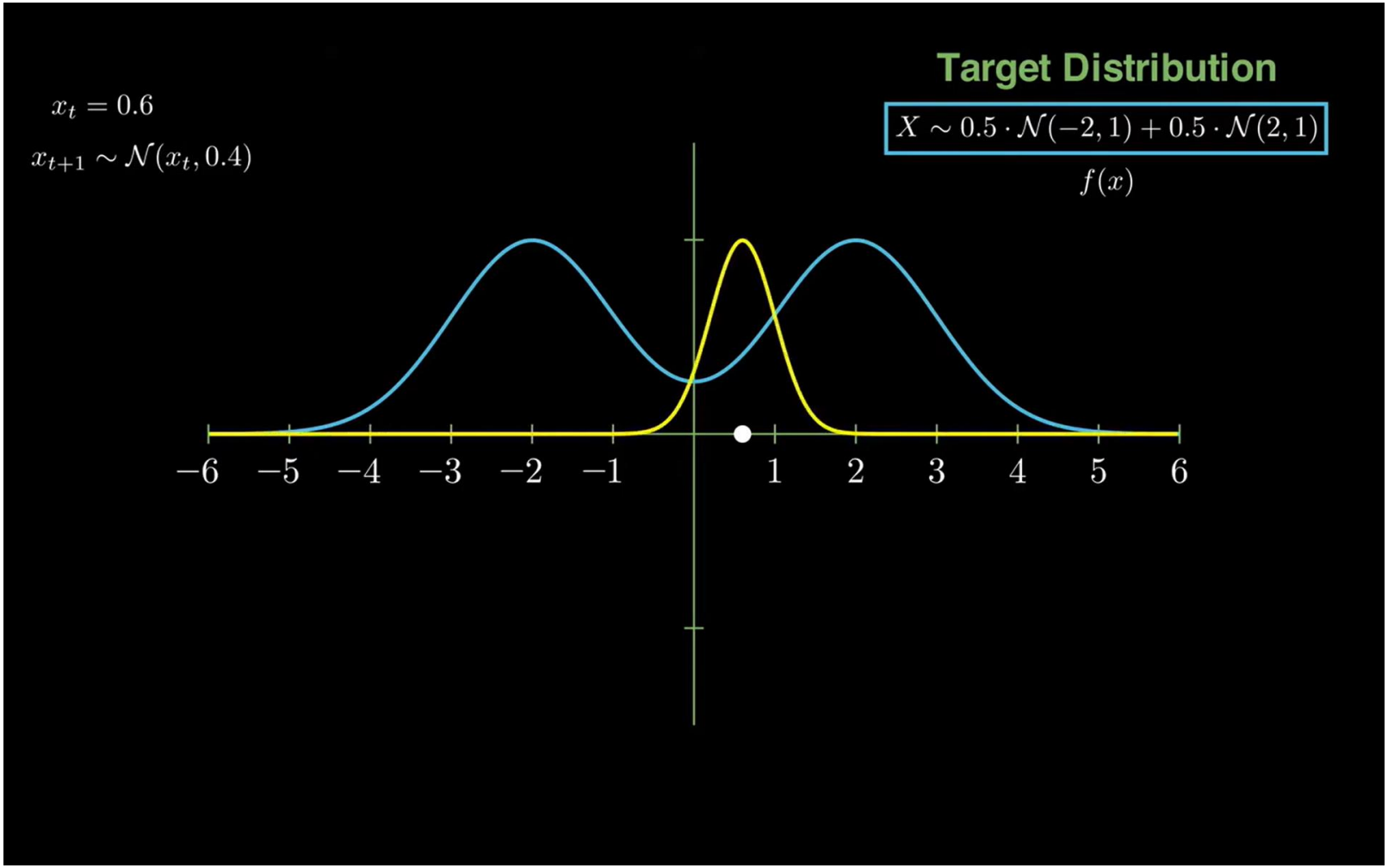


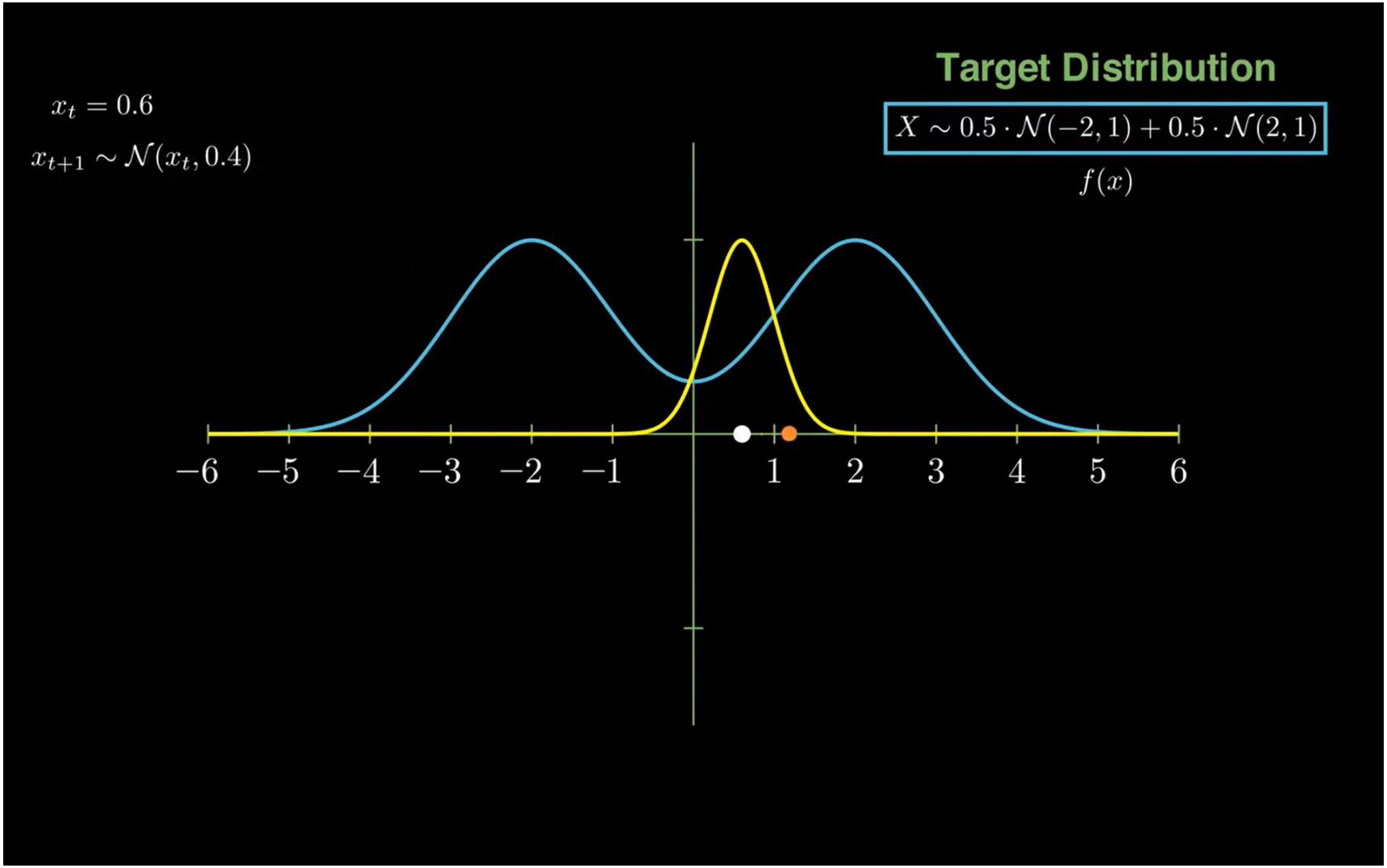


The proposal distribution could typically be a normal distribution centered on the current state x_n


Target Distribution
$X \sim 0.5 \cdot \mathcal{N}(-2, 1) + 0.5 \cdot \mathcal{N}(2, 1)$
$f(x)$
−6 −5 −4 −3 −2 −1 1 2 3 4 5 6


$x_t = 0.6$
Target Distribution
$X \sim 0.5 \cdot \mathcal{N}(-2, 1) + 0.5 \cdot \mathcal{N}(2, 1)$
$f(x)$
−6 −5 −4 −3 −2 −1 1 2 3 4 5 6


$x_t = 0.6$
$x_{t+1} \sim \mathcal{N}(x_t, 0.4)$
Target Distribution
$X \sim 0.5 \cdot \mathcal{N}(-2, 1) + 0.5 \cdot \mathcal{N}(2, 1)$
$f(x)$
−6 −5 −4 −3 −2 −1 1 2 3 4 5 6


$x_t = 0.6$
$x_{t+1} \sim \mathcal{N}(x_t, 0.4)$
Target Distribution
$X \sim 0.5 \cdot \mathcal{N}(-2, 1) + 0.5 \cdot \mathcal{N}(2, 1)$
$f(x)$
−6 −5 −4 −3 −2 −1 1 2 3 4 5 6

The accept-reject step

calculate the acceptance probability, $A(x_n \rightarrow x^*)$,

$$A(x_n \rightarrow x_*) = \min\left(1, \frac{p(x^*)}{p(x^n)} \times \frac{q(x_n|x^*)}{q(x^*|x_n)}\right)$$

ratio $p(x^*)/p(x_n)$ doesn't depend on the normalising constant for the distribution, basically a simple thing to calculate without any numerical integration

$q(x_n|x^*)/q(x^*|x_n)$ corrects for any biases that the proposal distribution might induce

Detailed balance

$$\forall a, b.$$

$$p(a)\,T(a \to b) = p(b)\,T(b \to a)$$

$$\frac{f(a)}{NC}\, g(b|a)\, A(a \to b) = \frac{f(b)}{NC}\, g(a|b)\, A(b \to a)$$

$$\frac{A(a \to b)}{A(b \to a)} = \frac{f(b)}{f(a)} \times \frac{g(a|b)}{g(b|a)}$$

r_g

r_f

$$r_f\, r_g < 1 \rightarrow \begin{cases} A(a \to b) = r_f\, r_g \\ A(b \to a) = 1 \end{cases}$$

$$r_f\, r_g \geq 1 \rightarrow \begin{cases} A(a \to b) = 1 \\ A(b \to a) = \dfrac{1}{r_f\, r_g} \end{cases}$$

$$A(a \to b) = \text{Min}(1, r_f\, r_g)$$

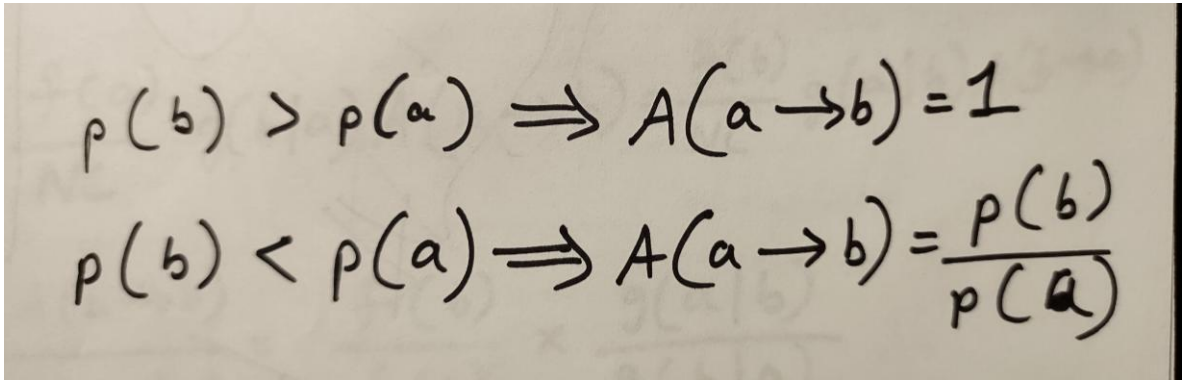
Now let the distributions be symmetrical

Let g be symmetric

$$A(a \to b) = \text{Min}(1, r_f) = \text{Min}\left(1, \frac{f(b)}{f(a)}\right)$$

$$p(b) > p(a) \Rightarrow A(a \to b) = 1$$

$$p(b) < p(a) \Rightarrow A(a \to b) = \frac{p(b)}{p(a)}$$



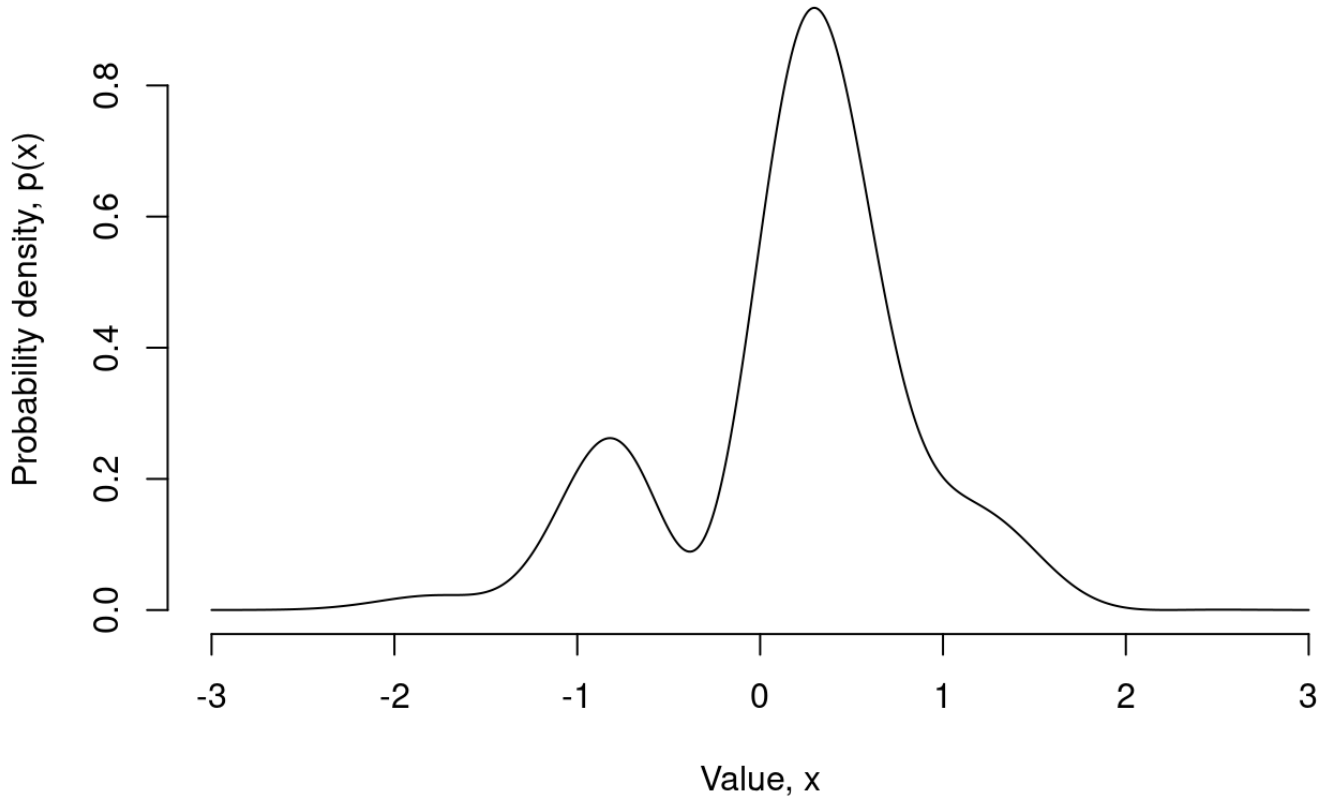
If acceptance ratio is <1, we cannot immediately reject b,

Even though f(b) is small, it still exist a chance to be drawn

Instead, we will generate a random variable u from Unif(0,1)
if $\alpha \geq u$, we accept b, otherwise, we reject b

Implementing the sampler

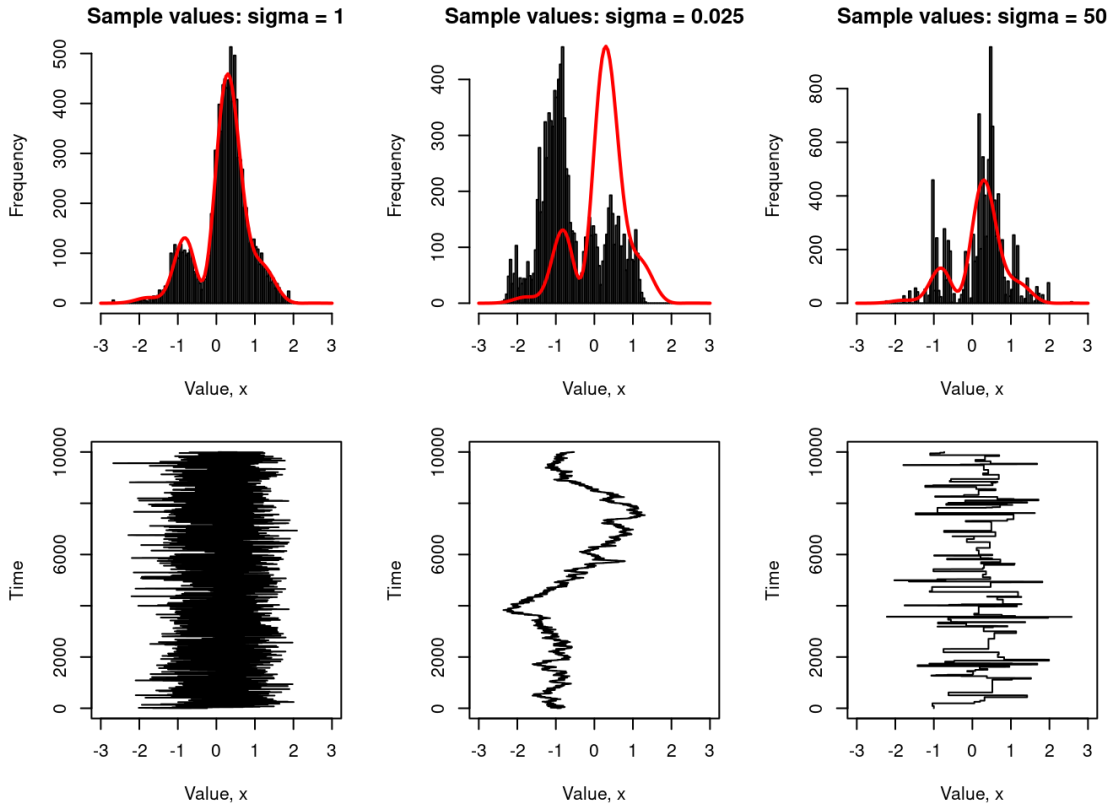
Let's head over to R studio



The effect of proposal width

If it's too narrow, your sampler will have an extremely high acceptance rate on average, but it will move around extremely slowly - very low mixing rate

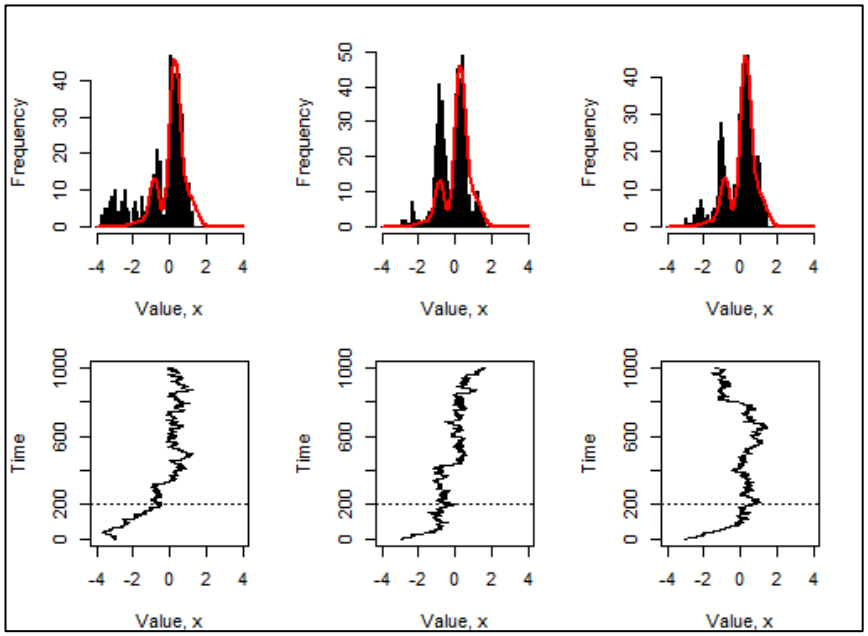
If it's too wide, the acceptance rate becomes too low and the chain gets stuck on specific values for long periods of time

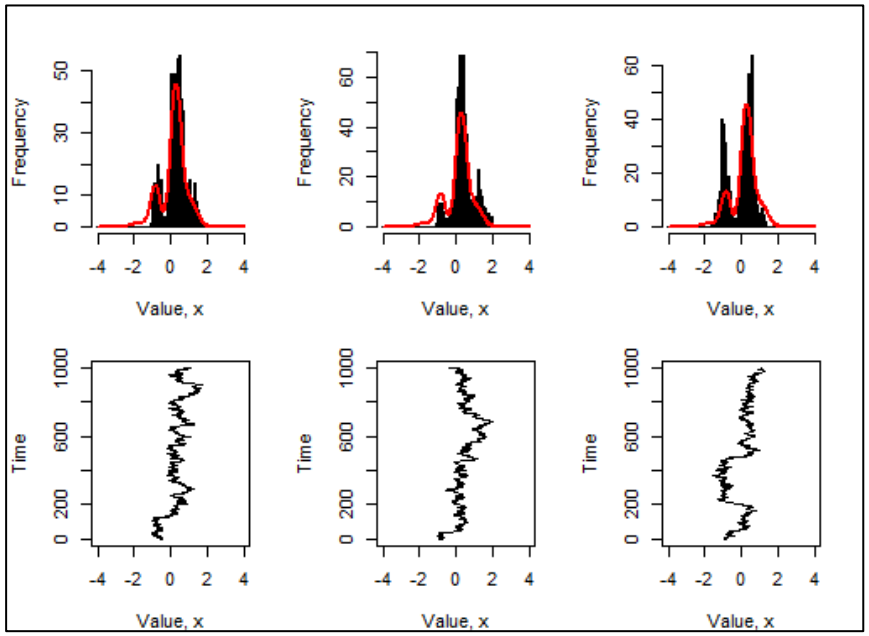


"play around" with the choice of proposal distribution

The role of the burn-in period

Let's work with a proposal distribution that is too narrow, say sigma = 0.1





histograms are biased towards the left (i.e., towards the bad start location)

Setting the burnin to 200

The role of the lag parameter

allow several iterations of the sampler to elapse in between successive samples; sigma = 50